

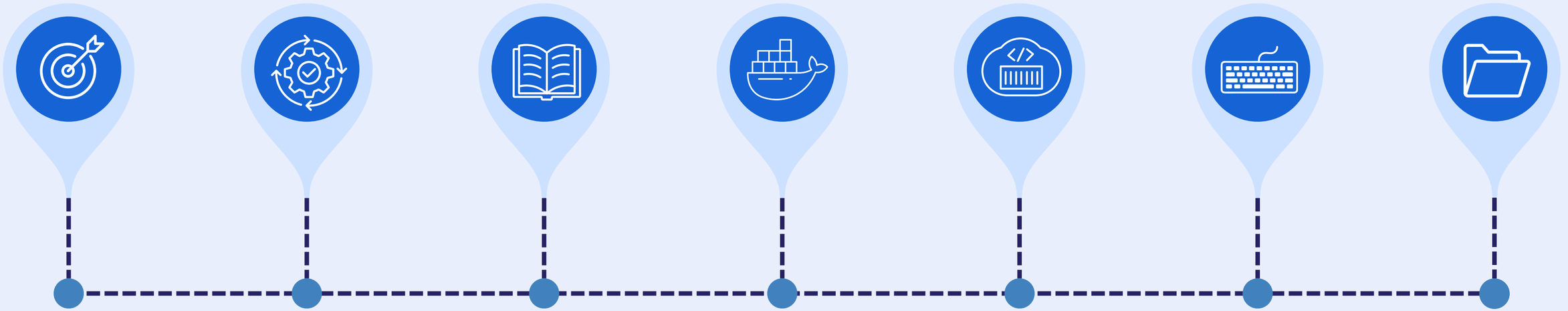
Apresentando:

DOCKER



Apresentado por: **Ana, Eduarda, Lorena e Pedro**

O que vamos ver?



O problema

Ambientes de Execução

Contexto histórico do Docker

Docker e seus conceitos

Docker nos ambientes

Demonstração prática

Persistência

O problema

Por que isso acontece??



Possíveis motivos

VERSÕES

Diferença na versão da linguagem utilizada para o desenvolvimento

DEPENDÊNCIAS

Bibliotecas necessárias, mas ausentes no outro computador.

BANCO

Utilização de bancos com configurações e versões diferentes

CONFIGURAÇÕES

Variáveis de ambiente e caminhos de arquivos.

SISTEMA OPERACIONAL

Diferenças entre sistemas operacionais

Versões da linguagem

Versões

Diferença na versão da linguagem utilizada para o desenvolvimento.

Exemplo

Ambiente A	Ambiente B	Resultado
Python 3.15	Python 3.8	Incompatibilidade

Dependências

Dependências

Bibliotecas necessárias, mas ausentes no outro computador.

Exemplo

Ambiente A	Ambiente B	Resultado
Pandas instalado	Pandas ausente	Módulo não encontrado

Banco de Dados

Banco

Utilização de bancos com configurações e/ou versões diferentes.

Exemplo

Ambiente A	Ambiente B	Resultado
Sintaxe MySQL	Sintaxe SQLite	Consulta incompatível

Configurações do Sistema

Configurações

Variáveis de ambiente e caminhos de arquivos.

Exemplo

Ambiente A	Ambiente B	Resultado
DB_HOST = endereço_banco DB_USER = admin DB_PASSWORD = definida	DB_HOST = ausente DB_USER = ausente DB_PASSWORD = ausente	Falha na conexão

Sistema Operacional

Sistema Operacional

Diferenças entre Sistemas Operacionais

Exemplo

Ambiente A	Ambiente B	Resultado
Windows C:\dados\arquivo.csv	Linux /home/dados/arquivo.csv	O caminho não é encontrado

Portanto, o problema pode estar nas **diferenças** entre os **ambientes** onde o código é executado.



O que é Ambiente de Execução?

É o conjunto de recursos de software e configurações necessários para que uma aplicação seja executada, como:

- Interpretador da Linguagem
- Bibliotecas e Dependências
- Sistema Operacional
- Configurações
- Variáveis de Ambiente

Tipos de Ambientes de Execução



01

Desenvolvimento

Ambiente em que ocorre o desenvolvimento da aplicação/produto

02

Homologação

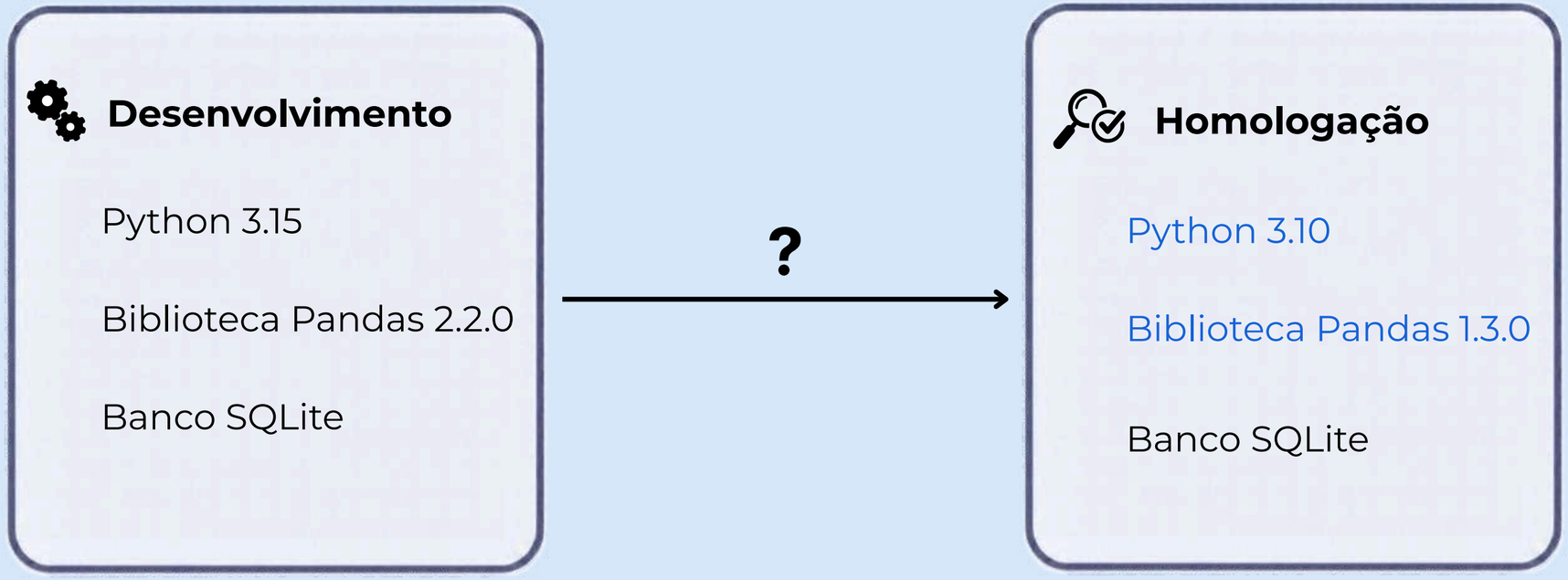
Ambiente em que ocorrem as validações

03

Produção

Ambiente em que a aplicação é disponibilizada para os usuários

Quando os ambientes não são consistentes



O que precisamos então?

Padronização

Usar um formato único e universal de empacotamento

Consistência

Apresentar um comportamento previsível nos diferentes ambientes.

Isolamento

Reduzir conflitos entre aplicações, dependências e recursos.

Reprodutibilidade

Conseguir recriar o mesmo ambiente, do zero, a qualquer momento, a partir das mesmas instruções.

Contexto histórico

Surgimento do Docker

Origem

Projeto desenvolvido na dotCloud.

Março de 2013

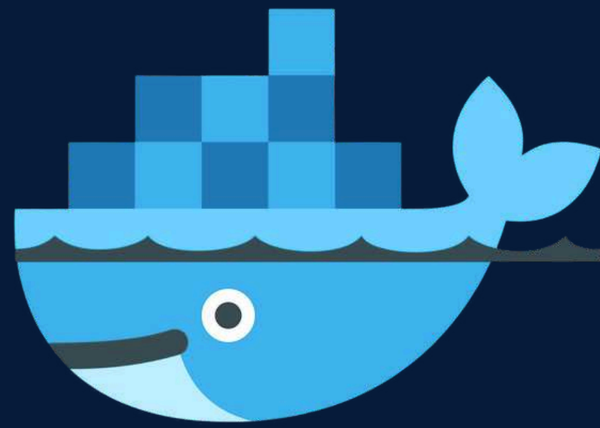
Apresentação pública do Docker na conferência Pycon.



Solomon Hykes · dotScale · 2013

O que é Docker?

Docker é uma plataforma de conteinerização que permite empacotar aplicações e suas dependências em containers, facilitando a execução em diferentes ambientes.



Empacota aplicações

Inclui dependências

Facilita a execução em diferentes ambiente

Imagem Docker

modelo usado para criar containers.



Imagem

Modelo de somente leitura que reúne os arquivos, o ambiente de execução e as dependências necessárias para executar uma aplicação.



Docker Hub

Docker Hub é um serviço online que permite encontrar, baixar e compartilhar imagens Docker, tanto públicas quanto privadas.

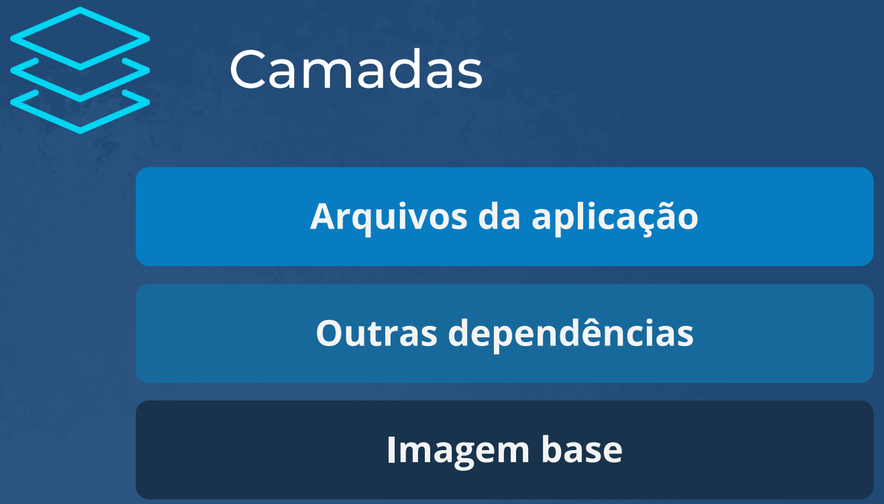


Docker Hub → docker pull →  **Imagem local** → docker run →  **Container**

Imagens Docker: camadas, tags e versões

como são formadas e identificadas?

Camadas



As imagens são formadas por camadas que podem ser reaproveitadas.

Tags e versões

nginx:alpine
Imagem Nginx · variante Alpine

python:3.12-slim
Python 3.12 · variante slim

A tag identifica uma versão ou variante da imagem.

Camadas ajudam no reaproveitamento; tags ajudam a identificar qual imagem utilizar.

Contêineres do Docker

a instância criada a partir de uma imagem.



Container

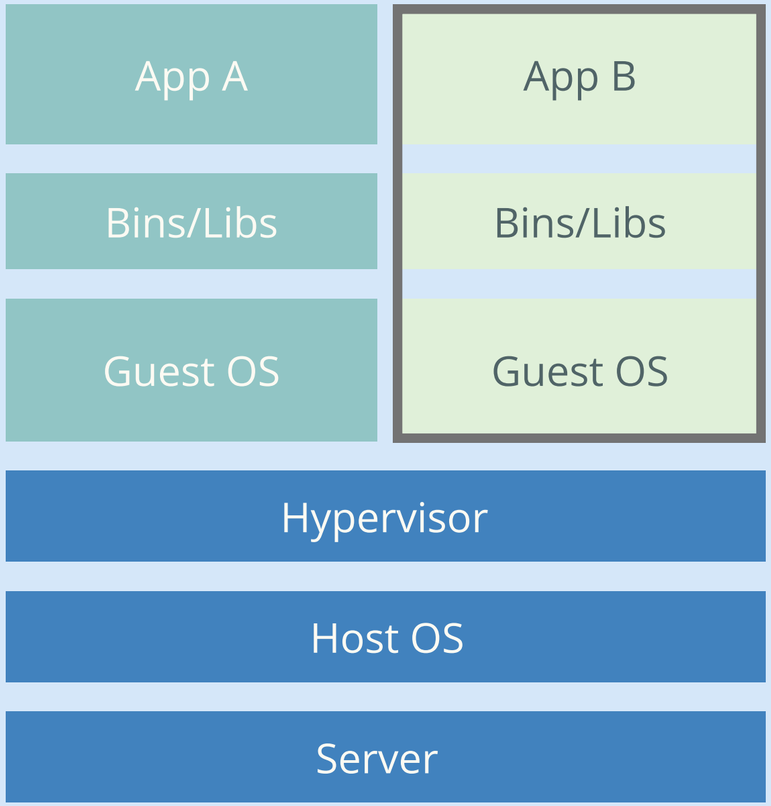
Ambiente isolado criado a partir de uma imagem, onde a aplicação é executada com seus arquivos e dependências. Pode ser iniciado, parado e removido.



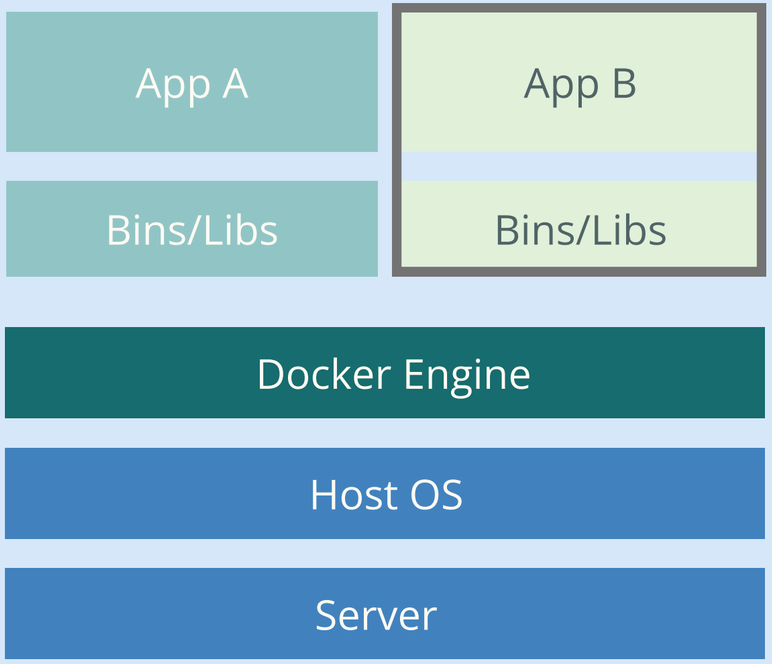
Docker Engine

É o componente responsável por gerenciar as operações do Docker, permitindo construir imagens e criar, iniciar e parar containers.

Máquinas virtuais x Containers



Máquina Virtual (VM)

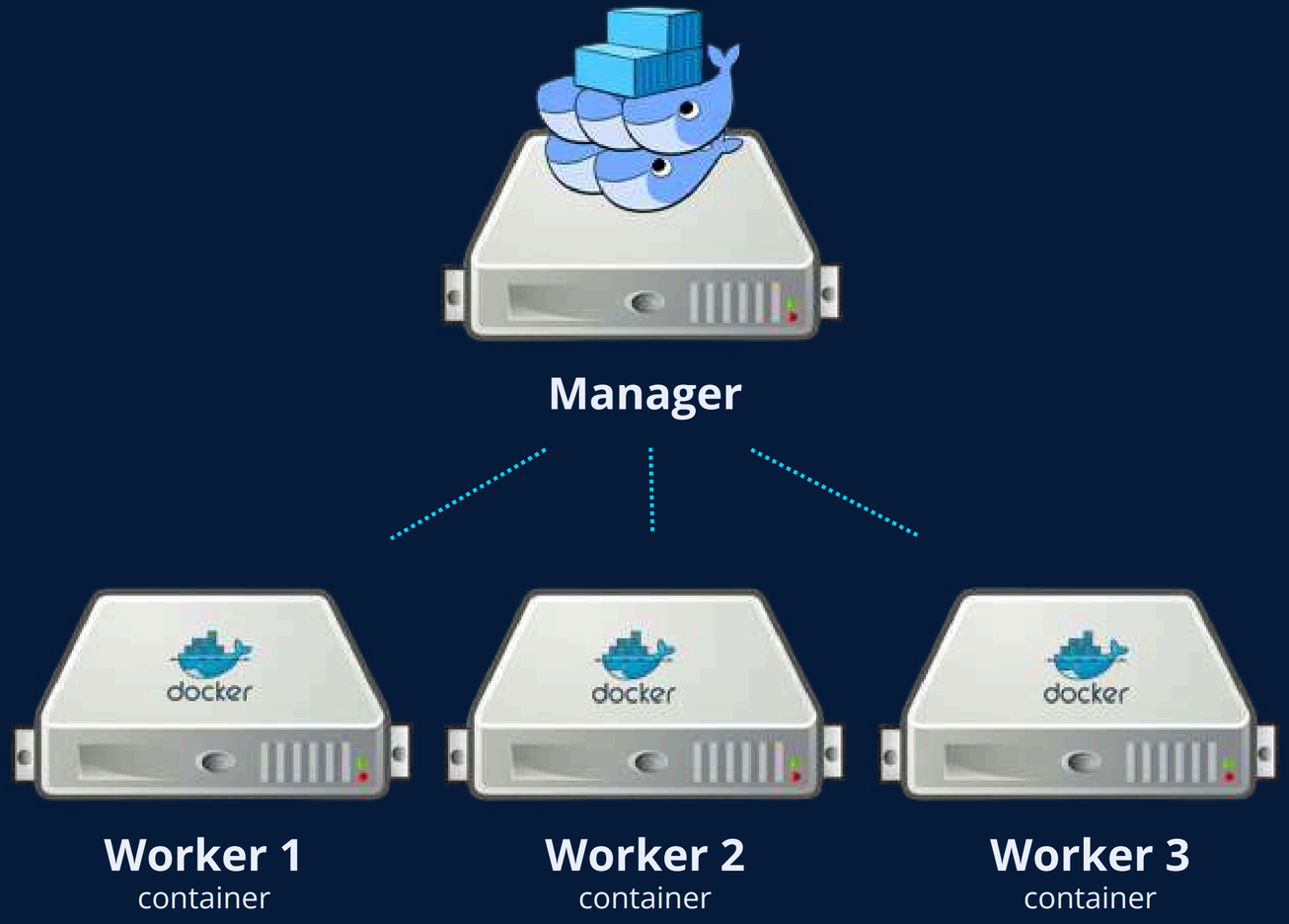


Containers

Docker Swarm

gerenciamento de containers em várias máquinas

É uma ferramenta nativa de orquestração do docker. Permite que containers executem distribuídos em em um cluster, controlando a quantidade de containers, registro, deploy e update de serviços.



Como o docker funciona?



Dockerfile

Define a configuração da imagem.





Imagem

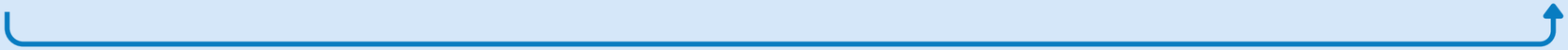
Contém o sistema, app e dependências





Container

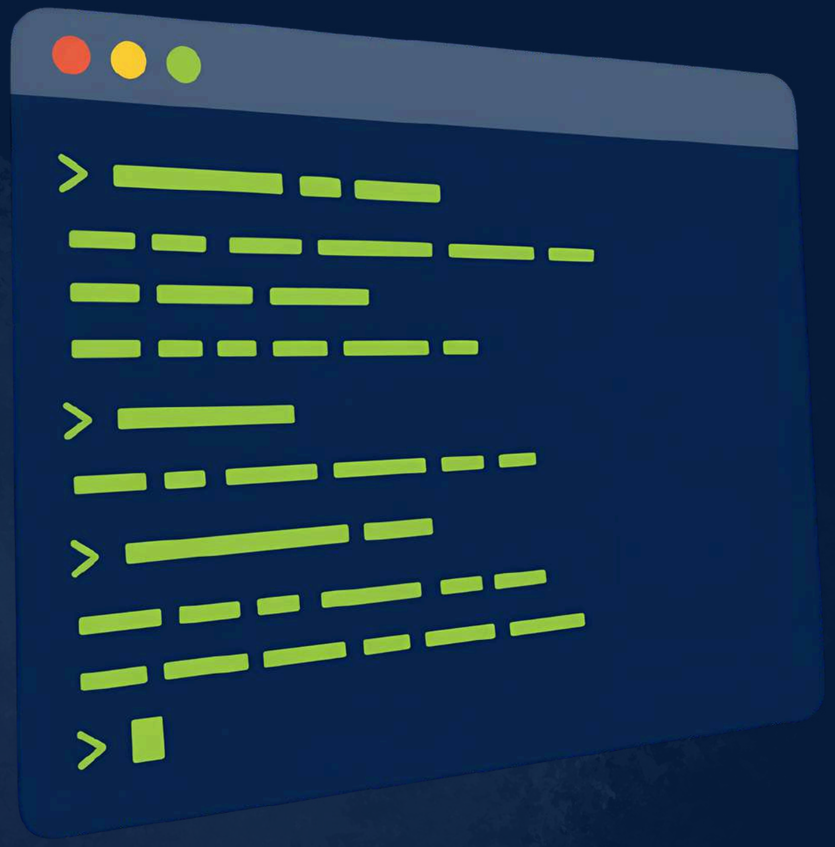
A imagem em execução.





Docker Engine gerencia todo o processo

Comandos básicos



docker pull	Baixa uma imagem.
docker images	Lista as imagens.
docker ps	Lista os containers em execução
docker run	Cria e inicia um container
docker stop	Para um container em execução.
docker build	Constrói uma imagem.

Criando uma imagem

1 Criamos os arquivos necessários.



2 Definimos as instruções no Dockerfile.

Dockerfile

```
# Usa a imagem oficial do Nginx
FROM nginx:alpine

# Copia o arquivo da nossa página
COPY index.html /usr/share/nginx/html/
```

3 Construímos a imagem.

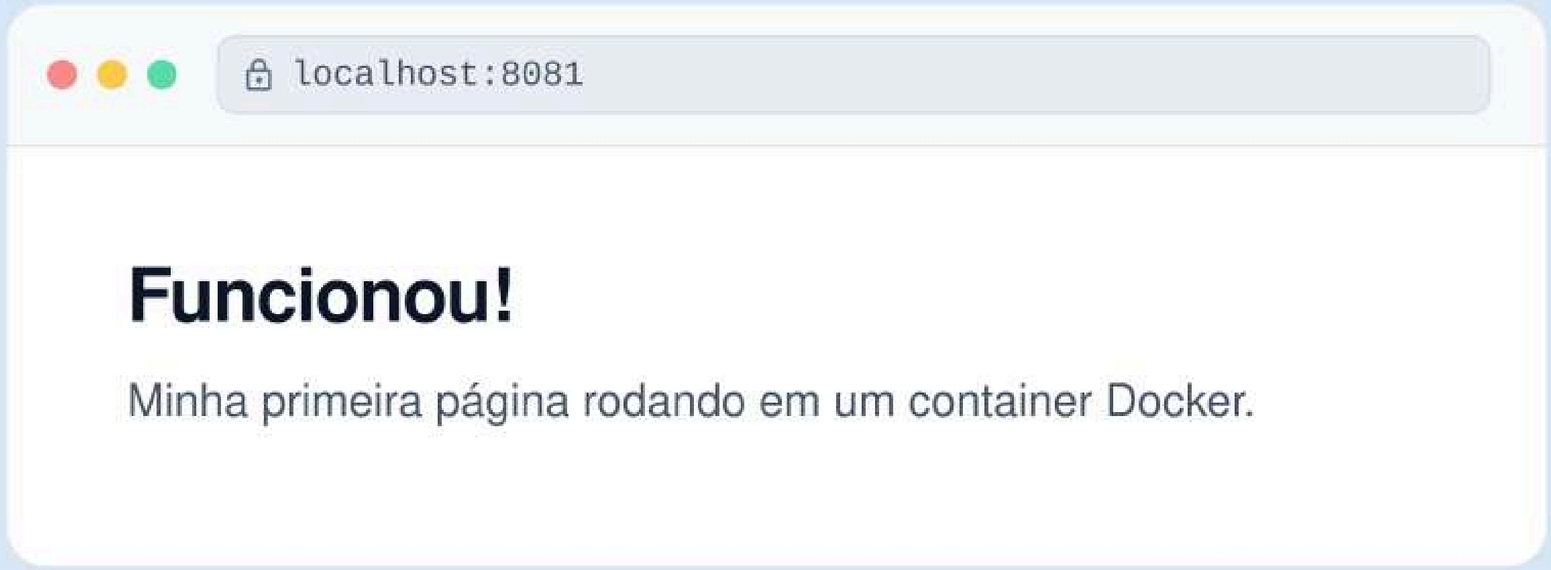


Executando o container

4 **Utilizamos a imagem criada para iniciar a aplicação em um container.**

```
>_ terminal

docker run -d --name meu-site-container -p 8081:80 meu-site
```



A aplicação está acessível pelo navegador.



Docker nos ambientes

Docker nos ambientes



Desenvolvimento

Fábrica: a imagem nasce na sua máquina.



Homologação

Testes em solo: a app:1.4.0 é validada.



Produção

Órbita: a mesma app:1.4.0 com usuários.



O módulo que voa é o mesmo que foi testado

O que muda em cada ambiente

	Desenvolvimento	Homologação	Produção
Imagem	construída aqui	baixada: app:1.4.0	a mesma app:1.4.0
Código	pasta do seu PC	dentro da imagem	dentro da imagem
Configuração	LOG_LEVEL=debug	LOG_LEVEL=info	LOG_LEVEL=warn
Dados	fictícios	de teste	reais
Portas	várias abertas	8080	só a 443

> Versão: a mesma tag, app:1.4.0, do desenvolvimento à produção

Construa uma vez

1. Build único

docker build gera a app:1.4.0 uma única vez

2. Publica no registro

docker push envia a imagem ao Docker Hub

3. Promove a mesma tag

homologação e produção só fazem docker pull



O que foi testado é o que vai para produção

Um Dockerfile, vários estágios

Dockerfile

```
FROM node:20-alpine AS base
WORKDIR /app
COPY package*.json ./

FROM base AS dev
RUN npm install
CMD ["npm", "run", "dev"]

FROM base AS prod
RUN npm ci --omit=dev
COPY . .
USER node
CMD ["node", "server.js"]
```

- **base**
  O que é comum: Node e o package.json
- **dev**
  Ferramentas e recarregamento automático
- **prod**
  Enxuta e sem rodar como root

Na hora do build, escolha o estágio:

```
docker build --target dev
docker build --target prod
```

Configuração de fora

Desenvolvimento

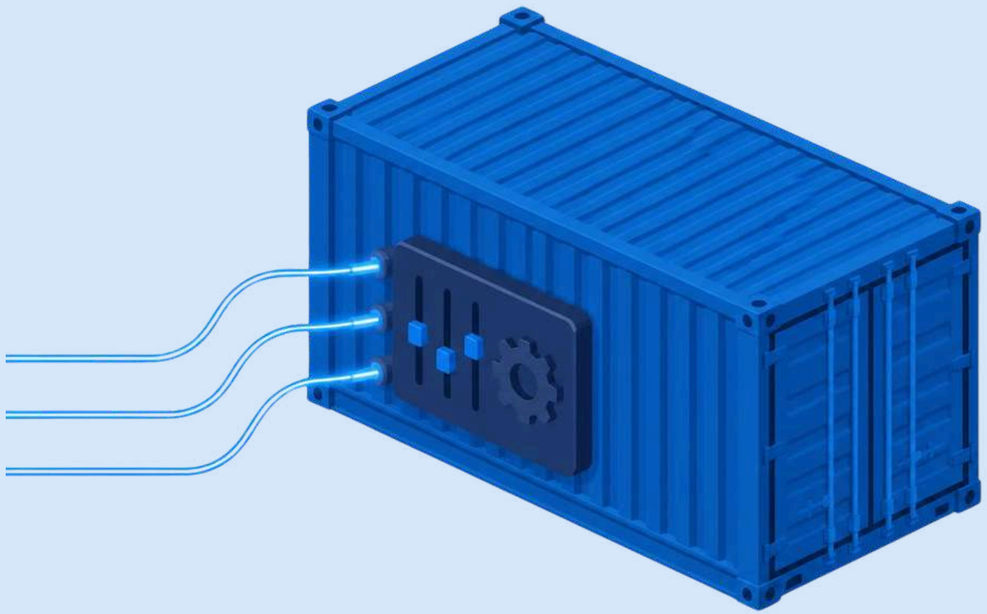
EMAIL_MODE=falso
DATABASE_URL=postgres://localhost

Homologação

EMAIL_MODE=equipe
DATABASE_URL=postgres://db-teste

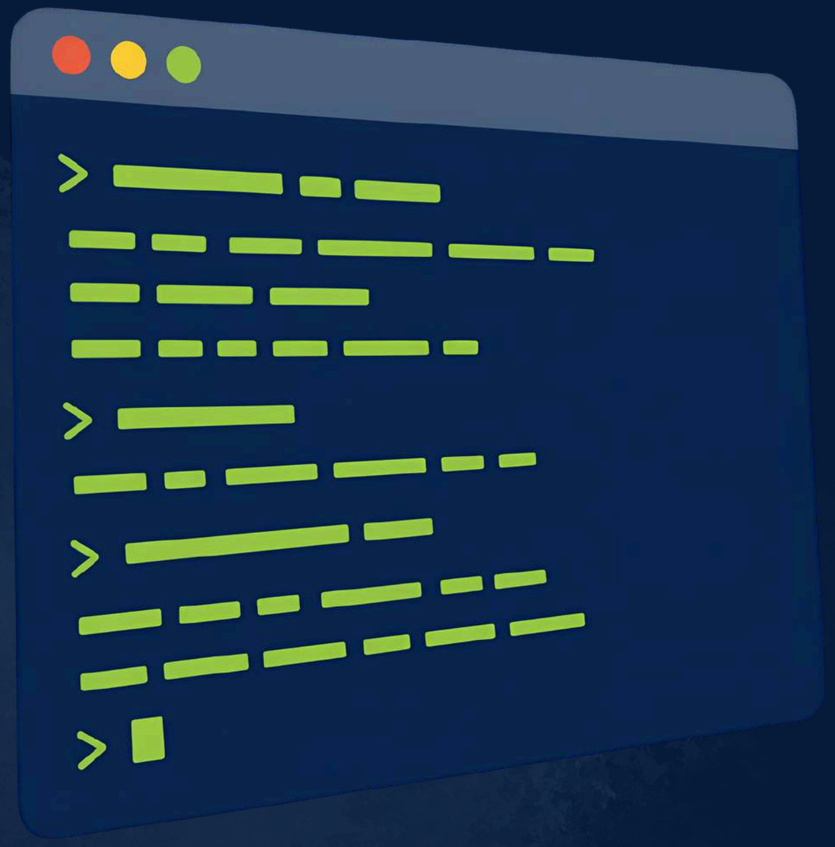
Produção

EMAIL_MODE=real
DATABASE_URL=secreta, fora do Git



A mesma imagem lê a configuração do ambiente

O arquivo .dockerignore



node_modules	Reinstalado no build, não copiado
.git	O histórico fica fora da imagem
.env	Senhas nunca entram na imagem
*.log	Logs de execução ficam de fora
coverage	Relatórios de teste ficam de fora

Resultado: imagem menor e build mais rápido

Código no desenvolvimento

Sem bind mount nem Compose Watch

1. Edita o código
2. Roda docker build de novo
3. Reinicia o container
4. Só então testa

Cada mudança vira um build novo

Com bind mount ou Compose Watch

1. Edita o código
2. Salva o arquivo
3. O container já vê a mudança

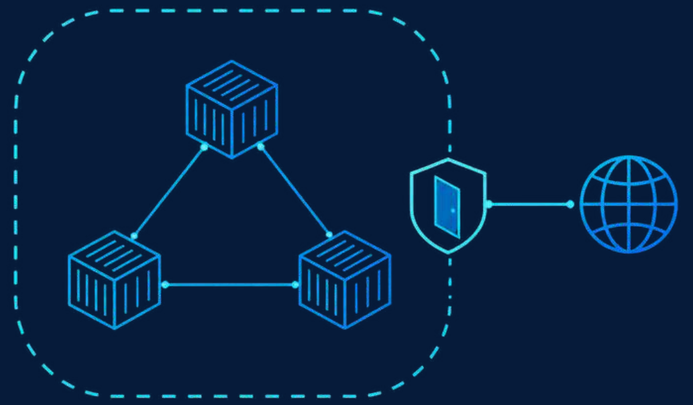
```
# Opção 1: bind mount (liga a pasta)
volumes:
  - ./src:/app/src
```

```
# Opção 2: Watch (copia ao salvar)
docker compose up --watch
```

Fora do dev não tem bind mount nem Watch: o código vai dentro da imagem

Redes e portas

Na rede interna, cada container acha o outro pelo nome: a aplicação acessa o banco em db:5432, sem IP fixo.



Rede interna entre os containers

Dev: portas abertas para inspecionar

Produção: só o proxy na porta 443

Banco de dados nos ambientes



Desenvolvimento
sobe em segundos, descartável

Homologação
mesma versão, dados de teste

Produção
volume com backup, ou banco gerenciado

Só a DATABASE_URL muda entre eles

Um Compose por ambiente

Desenvolvimento

compose.override.yaml

- NODE_ENV: development
- Memória: sem limite
- Reinício: manual

```
docker compose up --watch
```

+

Homologação

compose.homolog.yaml

- NODE_ENV: production
- Memória: 512 MB
- Reinício: se der erro

```
docker compose -f compose.yaml
-f compose.homolog.yaml up -d
```

+

Produção

compose.prod.yaml

- NODE_ENV: production
- Memória: 1 GB
- Reinício: sempre

```
docker compose -f compose.yaml
-f compose.prod.yaml up -d
```

+

compose.yaml: a base comum (serviços, rede e volume)

CI/CD e rollback



Analogias

Docker	Analogias
Aplicação	O bolo que queremos entregar
Ambiente de execução	A cozinha, utensílios e equipamentos
Dockerfile	A receita
docker build	Lê a receita e prepara a massa
Imagem	É a massa pronta, congelada e identificada por versão
docker run	Abre uma estação na cozinha e coloca uma cópia dessa massa em produção
container	A estação funcionando e servindo os bolos
docker compose	o gerente que coordena várias estações (cozinhas)
Volume	caderno de pedidos, onde os dados são persistidos
DockerHub	Um grande catálogo de massas e bases prontas



Vamos para a prática

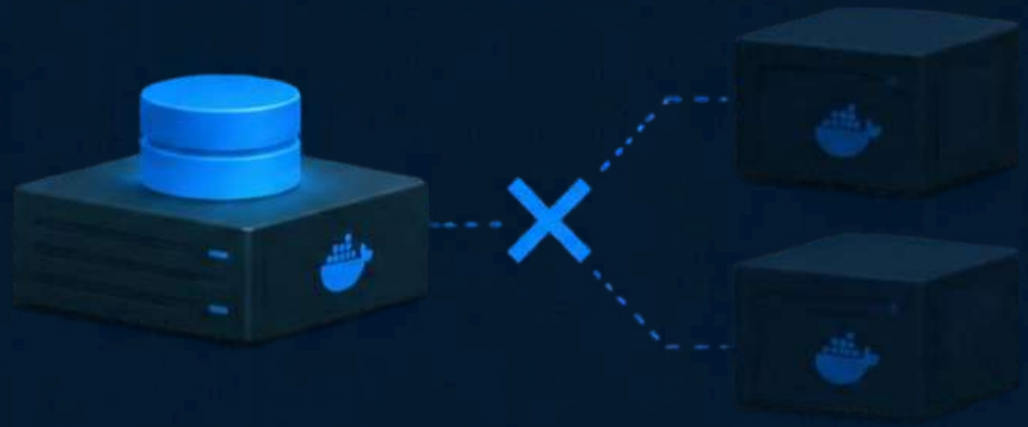
Persistência não é sincronização

VOLUME PRESERVA DADOS NO MESMO HOST DOCKER



os dados permanecem
seguros no mesmo ambiente

VOLUME NÃO COPIA DADOS PARA OUTROS COMPUTADORES



os dados não são enviados
para outros hosts

Volumes de computadores diferentes não são sincronizados automaticamente.

Cada computador possui seu volume



Executar o mesmo Compose **não significa compartilhar** o mesmo volume.

Dados compartilhados



Usuários
acessam pelo navegador

A aplicação
recebe as requisições

PostgreSQL
centraliza os dados

Para compartilhar dados, todos acessam a mesma aplicação e o mesmo banco

Vantagens, limitações e segurança



Vantagens

- Mesmo ambiente em máquinas diferentes
- isolamento entre aplicações
- leve e rápido de criar
- facilita testes e atualizações



Limitações

- Exige conhecimento prévio
- Persistência e backups precisam ser planejados
- Consome recursos
- Não substitui monitoramento



Segurança

- Não compartilhar senhas
- Manter imagens e dependências atualizadas
- usar poucos privilégios e expor somente o necessário
- Autenticação, HTTPS e backups


Obrigado pela atenção!